

Robotique
CRC
Robotics

COMPÉTITION DE PROGRAMMATION



2027

PROBLÈME PRÉLIMINAIRE 1

QUELQUES NOTES

- Les règles complètes sont dans la section 4 du livret des règlements.
- Vous avez jusqu'au **vendredi 13 novembre, 23h59** pour remettre votre code.
- N'hésitez pas à utiliser le forum de programmation sur le discord de la CRC pour poser vos questions et discuter des problèmes. Il est là pour ça!
- **On vous donne des fichiers modèles faciles à utiliser pour votre code et pour faire vos tests. Vous devez les utiliser pour résoudre le problème!**

UTILISATION DU FICHIER MODÈLE

- Le fichier de test pour chaque défi contient l'endroit pour écrire votre solution ainsi que les tests. Pour rouler les tests dans un éditeur comme VSCode, vous n'avez seulement qu'à cliquer sur le bouton ▸ ou sinon rouler le fichier avec python. Il y aura aussi une version Jupyter des fichiers si vous préférez les utiliser.
- Les points mis dans le document indiquent la difficulté et le pointage attribué pour la réussite pour chaque défi. Ce problème préliminaire aura une valeur globale de 2% du défi principal.

STRUCTURE

Une petite mise en situation comme celle-ci explique les fondements de chaque défi et offre les bases nécessaires pour résoudre celui-ci.

Spécification d'entrée et de sortie:

Contient les caractéristiques des entrées fournies ainsi que les critères attendus pour les sorties du programme.

Exemple d'entrée et de sortie:

Contient un exemple d'entrée, parfois constitué lui-même de plusieurs sous-exemples, pour que vous puissiez tester votre programme. Chaque exemple de sortie donne la réponse attendue pour l'entrée correspondante.

Explication de la première sortie:

Décortique davantage le défi en expliquant comment la première entrée est traitée et en montrant le chemin menant à cette réponse.

Biscuits!

Rien de mieux que quelques biscuits pour commencer la journée ! Ils sont tellement délicieux qu'ils peuvent transformer un lundi matin plate à 5 h en véritable célébration. Il y a tellement de choses à dire sur ces délicieux petits cercles qu'on va plonger dans quelques-uns de nos sujets préférés à propos des biscuits ! 🍪

Partie 1: Les cookies sur internet (10 points)

Les biscuits sont merveilleux lorsqu'on parle de nourriture, mais les cookies sont aussi un concept très important sur Internet. Des cookies sont créés sur presque tous les sites Web pour vous éviter d'avoir à vous reconnecter chaque fois ou pour mémoriser certaines de vos préférences. Ils ont de nombreuses utilisations, mais aujourd'hui, nous allons nous concentrer sur la recherche et la création de cookies.

Nous allons créer un cookie personnalisé pour chaque utilisateur qui en fait la demande et qui n'en possède pas déjà un. Si l'utilisateur possède déjà un cookie, nous lui retournerons celui-ci ; sinon, nous en créerons un nouveau. Pour créer un cookie unique, nous utiliserons les cinq premières lettres du nom d'utilisateur, suivies du texte aléatoire qui vous est fourni. Les cinq premières lettres du nom d'utilisateur seront différentes pour chaque utilisateur.

Spécification d'entrée et de sortie:

Comme entrée vous recevrez :

cookies : La liste des cookies existant

username: Le nom d'utilisateur de l'utilisateur voulant un cookie

random_text: Le texte aléatoire à utiliser si vous devez générer un nouveau cookie

Comme sortie, vous devrez fournir le cookie de l'utilisateur soit de la liste fournie ou sinon celui que vous avez créé.

Exemple d'entrée:

```
cookies = [“frankRCdsl3csW8”, “rando3J5a9icd0s”]
username = biggest_crc_fan
random_text = 4mvdo1fe2nf833c
```

```
cookies = [“frankRCdsl3csW8”, “rando3J5a9icd0s”]
username = frank_crc
random_text = 4mvdo1fe2nf833c

cookies = [“frankRCdsl3csW8”, “rando3J5a9icd0s”]
username = random_test
random_text = 4mvdo1fe2nf833c
```

Exemple de sortie:

```
bigge4mvdo1fe2nf833c
frankRCdsl3csW8
rando3J5a9icd0s
```

Explication de la première sortie:

Dans le premier exemple, nous prenons les cinq premières lettres du nom d'utilisateur et obtenons « bigge ». Nous examinons la liste des cookies qui nous sont fournis. Comme aucun d'entre eux ne commence par le nom de notre utilisateur, cela signifie que nous devons lui créer un nouveau cookie. Nous combinons les cinq premières lettres avec le texte aléatoire pour obtenir le cookie final.

Dans le deuxième exemple, nous prenons les cinq premières lettres du nom d'utilisateur et obtenons « frank ». Nous examinons la liste des cookies et constatons qu'il y en a un qui commence par « frank ». Nous récupérons ce cookie dans la liste et le retournons à l'utilisateur, puisqu'il possède déjà un cookie valide !

Partie 2: Vente de biscuits (10 points)

Nous voulons organiser une vente de biscuits pour apporter plus de budget pour la compétition à venir. Nous voulons savoir à partir de combien de biscuits vendus nous commençons à faire du profit et combien de profits final après la vente de la journée.

Vous devez donc déterminer combien de biscuits doivent être vendus pour commencer à faire du profit et ensuite le profit à la fin de la journée. Ils seront tous au même prix de vente pour la journée, pour faciliter la tâche et il va toujours y avoir au moins une quantité de biscuit pour avoir du profit.

Spécification d'entrée et de sortie:

En entrée vous recevrez:
c : le coût dépensé pour faire les biscuits *int*
p : le prix des biscuits à la ventes *float*
q : la quantité de biscuit de la journée. *int*

En sortie vous devrez donner une *list* de deux *int* désignant le nombre de biscuits vendu pour commencer à faire du profit et ensuite le profit de la journée en dollars.

Exemple d'entrée:

```
c = 20 p = 1.0 q = 160
c = 51 p = 1.50 q = 150
c = 35 p = 2.50 q = 68
c = 50 p = 0.50 q = 102
```

Exemple de sortie:

```
[20, 140]
[34, 174]
[14, 135]
[100, 1]
```

Explication de la première sortie:

Il faut 20 biscuits pour commencer à faire du profit car chaque biscuit est vendu à 1$ chaque et les biscuits coûtent 20$ pour les faire ensuite il reste 140 biscuits à 1$ chaque. Et cela donne 140$ de profit pour cette vente.

Partie 3: Recette de biscuit (30 points)

Vous avez acheté un livre de recette qui contient tous les ingrédients et les étapes nécessaires pour préparer un seul biscuit. Il y a des membres de votre famille qui vont venir chez vous pour une fête d'Halloween et vous aimeriez préparer suffisamment de biscuits pour tout le monde. Vous souhaitez disposer des ingrédients nécessaires, soit la quantité de biscuits que vous désirez préparer. Voici les instructions que vous devez suivre faire :

- Multipliez les ingrédients nécessaires par le nombre de biscuits que vous voulez préparer, à l'exception du temps de cuisson et de la température du four ; voir ci-dessous.
- Ne modifiez aucun *texte* dans les instructions fournies ; vous devez uniquement modifier les quantités.
- N'augmentez pas la température du four ; vous ne voulez pas brûler votre maison! Dans l'instruction de température il y aura toujours le mot "oven".
- Pour la durée de la cuisson, vous devrez ajouter une minute de plus pour chaque biscuit à partir du 6e. L'instruction de la durée de cuisson va toujours contenir le mot "minutes".

Spécification d'entrée et de sortie:

En entrée vous recevrez une liste avec des *strings* qui contiennent les ingrédients et les étapes pour la préparation des biscuits, ainsi que le nombre de biscuits que vous voulez préparer.

En sortie, vous donnerez la nouvelle version de la liste des étapes et des ingrédients multipliés par le nombre de biscuits à cuire.

Exemples d'entrée:

```
num_of_cookies = 6
recipe = [
   "Preheat the oven to 375 degrees fahrenheit",
   "Add 80 grams of flour into the bowl",
   "Add in 40 grams of brown sugar",
   "Incorporate 30 grams of softened butter into the bowl",
   "Blend in 2 eggs",
   "Mix the ingredients until the mixture is smooth",
   "Pour the mixture into a tray then bake for 12 minutes"
]
```

```
9,
[
"Preheat the oven to 375 degrees fahrenheit",
"Add in 35 grams of sugar",
"Add 75 grams of wheat flour into the bowl",
"Incorporate 25 grams of softened butter into the bowl",
"Blend in 1 eggs",
"Mix the ingredients until the mixture is smooth",
"Pour the mixture into a tray then bake for 11 minutes",
],
```

Exemples de sortie:

```
[“Preheat the oven to 375 degrees fahrenheit”,
“Add 480 grams of flour into the bowl”,
”Add in 240 grams of brown sugar”,
“Incorporate 180 grams of softened butter into the bowl”,
“Blend in 12 eggs”,
”Mix the ingredients until the mixture is smooth”,
“Pour the mixture into a tray then bake for 13 minutes”]
```

```
[
"Preheat the oven to 375 degrees fahrenheit",
"Add in 315 grams of sugar",
"Add 675 grams of wheat flour into the bowl",
"Incorporate 225 grams of softened butter into the bowl",
"Blend in 9 eggs",
"Mix the ingredients until the mixture is smooth",
"Pour the mixture into a tray then bake for 15 minutes",
],
```

Explication de la première sortie:

Dans le premier exemple, le nombre de grammes de farine, de cassonade et de beurre ramolli sont multipliés par la quantité de biscuits à cuire (multipliés par 6). La température du four reste inchangée.

Pour la durée de cuisson, nous avons ajouté 1 minute dans la durée normale de 12 minutes. Parce que nous avons un nombre de biscuits plus grand que 5 (6-5=1). Donc, nous avons une durée de 13 minutes. (*Si nous avions eu 7 biscuits, il aurait fallu ajouter 2 minutes)

Partie 4: Usine à biscuit (30 points)

Chef Pâtissiers Biskwi devait rendre une commande impeccable de cookies au patron de la multinationale Choco & Co. Le patron a une obsession malsaine à que tout soit **EXACTEMENT en ordre en tout temps**, donc pour les commandes Chef Biskwi a investi dans une machine qui lui permettait de remettre dans l'ordre croissant des numéros de boîtes de cookies juste avant d'envoyer la commande.

Mais aujourd'hui, le pire est arrivé. La machine a brisé pile au moment de la remise de la commande. C'EST LA CATASTROPHE. Cependant, la machine, avant de donner son dernier souffle, peut effectuer une dernière et unique intervention.

Voici comment fonctionne cette opération miracle : sur le tapis, il y a n boîtes, chacune portant un numéro différent entre 1 et n (chaque numéro n'apparaît qu'une seule fois: pas de doublons et pas de numéro manquant). Le bras peut :

- choisir n'importe quel groupe de positions sur le tapis (elles n'ont pas besoin d'être collées les unes aux autres. Le bras peut très bien attraper la 2e boîte, sauter la 3e, puis attraper la 4e, etc.) ;
- puis remettre les boîtes de ce groupe à leur place, mais **dans l'ordre inverse**. Autrement dit, la boîte la plus à gauche du groupe et la boîte la plus à droite du groupe échangent leur place, la 2e en partant de la gauche échange avec la 2e en partant de la droite, et ainsi de suite jusqu'au milieu.

Toutes les boîtes qui n'ont pas été choisies restent exactement où elles sont.

Chef Biskwi doit savoir, avant même d'essayer (le bras n'aura pas de deuxième chance, et une boîte mal replacée finira directement... à la poubelle), si une seule opération suffit à remettre tout le tapis en ordre croissant (1, 2, 3, ..., n).

Spécification d'entrée et de sortie:

En entrée, la ligne contient *n* nombres, l'ordre actuel (chaotique) des boîtes sur le tapis. Ces n nombres sont justement les numéros de 1 à n, chacun apparaissant une seule fois, mais placés dans le désordre.

En sortie pour chaque commande, retourner "YES" si Chef Biskwi peut sauver la livraison en une seule opération, ou "NO" si la commande est irrémédiablement perdue (et que Choco & Co. devra attendre... ou pire, annuler le contrat).

Exemple d’entrée:

[1]
[4, 2, 3, 1]
[3, 4, 1, 2]
[2, 1, 3, 5, 4]
[1, 6, 3, 4, 5, 2]
[1, 2, 3]
[2, 1]
[3, 2, 1]
[2, 3, 1]
[5, 2, 3, 4, 1]
[2, 1, 4, 3]

Exemple de sortie:

“YES”
“YES”
“NO”
“NO”
“YES”
"YES"
"YES"
"YES"
"NO"
"YES"
"NO"

Explication de la dernière sortie:

Le tapis affiche les boîtes 1, 6, 3, 4, 5, 2 dans cet ordre. Le bras peut attraper les boîtes aux positions 2, 4 et 6 (les boîtes numérotées 6, 4 et 2), il les redépose dans l’ordre inverse à ces mêmes positions : la boîte de la position 2 et celle de la position 6 échangent leur place, tandis que celle du milieu (position 4) reste où elle est puisqu’elle n’a personne avec qui échanger. Le tapis devient alors 1, 2, 3, 4, 5, 6. **Miracle !** La commande peut partir à temps.

Partie 5: Être au chocolat ou non, telle est la question (35 points)

Une pâte à biscuit est étalée sur le comptoir sous forme de liste, tout comme on ne place pas le chocolat un par un sur la pâte, mais qu'elle est mélangée à la pâte est placée de façon aléatoire dans la pâte, les pépites de chocolat représenter par la lettre C sont placés au hasard.

Une fois la pâte faite et étalée, il suffit de couper avec un emporte-pièce et mettre le tout au four pour avoir des biscuits. Ce qui est dommage avec cela, c'est qu'on peut se ramasser avec des biscuits sans chocolat, le drame !!!

Spécification d'entrée et de sortie:

Vous allez recevoir une *list* de *string* qui est une représentation d'une pâte à biscuit originellement formée de P, de façon aléatoire, des C vont être mis pour représenter les pépites de chocolat.

Vous allez recevoir une valeur de type int qui représente la longueur et la largeur de l'emporte pièce avec lequel il faut découper les biscuits. A noter qu'aucun reste de pâte sera créé, toute la pâte est utilisée.

En sortie, vous devrez donner le nombre de biscuits qui ne contiennent pas de chocolat.

Exemple d'entrée:

```
size = 2
[['P' 'P' 'P' 'P']
 ['P' 'C' 'C' 'P']
 ['C' 'P' 'C' 'P']
 ['P' 'C' 'C' 'C']]

size = 2
[['P' 'C' 'P' 'C' 'C' 'P' 'P' 'C']
 ['P' 'P' 'C' 'C' 'P' 'P' 'C' 'P']
 ['C' 'C' 'P' 'P' 'P' 'P' 'P' 'P']
 ['P' 'P' 'P' 'P' 'P' 'P' 'C' 'C']
 ['P' 'P' 'P' 'C' 'P' 'P' 'C' 'C']
 ['C' 'P' 'P' 'P' 'C' 'P' 'P' 'P']
 ['C' 'C' 'P' 'P' 'P' 'C' 'P' 'P']
 ['P' 'P' 'C' 'P' 'P' 'C' 'P' 'C']]
```

Exemple de sortie:

```
0
2
```

Explication de la première sortie:

Dans le premier exemple, nous avons une grille 4x4 avec un emporte pièce de grandeur 2 (donc un carré 2x2). Nous avons le morceau en haut à gauche qui est:
P P
P C
Nous avons donc au moins une pépite de chocolat. Pour le morceau en haut à droite, nous avons:
P P
C P
Il y a aussi au moins une pépite de chocolat. Pour les deux derniers biscuits (en bas à gauche et en bas à droite, nous avons une ou deux pépites de chocolat donc il y a un total de 0 biscuits sans pépites de chocolat!

Partie 6: Plaque de cuisson (40 points)

Dum-E, le bras robotique de Tony Stark, est en train d'apprendre à faire cuire des biscuits. Or, le bras a de la misère à arranger les biscuits sur la plaque de cuisson; votre défi est alors de créer un programme capable d'aider *Dum-E* à les placer.

Spécification d'entrée et de sortie:

En entrée, vous recevrez une liste de positions et tailles de biscuits (`[(x1, y1, rayon1), (x2, y2, rayon2), ...]`). En sortie, vous devez retourner un booléen validant ou non l'arrangement donné en entrée - il est valide si et seulement s'il y a guère que biscuits qui se touchent ou qui se chevauchent.

Exemple d'entrée:

1. `[(2, 3, 3)]`
2. `[(2, 3, 3), (5, 7, 1)]`
3. `[(2, 3, 3), (5, 7, 1), (2, 8, 2)]`
4. `[(2, 3, 3), (5, 7, 1), (9, 10, 5)]`

Exemple de sortie:

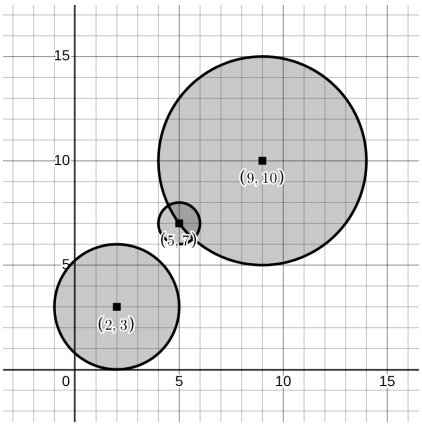
1. `True` - il n'y a qu'un seul biscuit sur la plaque de cuisson
2. `True` - il y a un **espace vide** d'une distance de 1 entre les deux biscuits
3. `False` - les premier et troisième biscuits **se touchent sur leurs bordures**
4. `False` - les deuxième et troisième biscuits **se chevauchent** l'un sur l'autre

Explication de la quatrième sortie:

Afin de vérifier la distance entre deux biscuits, il est possible d'utiliser pythagore (formule):

$$distance = \sqrt{(x1 - x2)^2 + (y1 - y2)^2}$$

Avec pythagore, nous pouvons calculer que la distance est de $\sqrt{(5 - 9)^2 + (7 - 10)^2} = 5$. Or, puisque la somme de leurs rayons est de $1 + 5 = 6$, il est possible de conclure qu'il y a chevauchement de 1 unité.



En utilisant le même raisonnement pour les autres combinaisons de biscuit, il est possible de remarquer qu'il n'y a pas de tel chevauchement. Or, puisqu'il y a eu **au moins un**, l'arrangement au complet est invalide et `False` est retourné.